

Cross-Architecture CKA Reveals Where Sequence Models Converge and Diverge on Synthetic Mechanistic Tasks

Anonymous Authors¹

Abstract

Mechanistic interpretability has largely progressed through architecture-specific case studies, especially in Transformers, while recurrent models are often evaluated only behaviorally or through formal-language theory. This leaves a basic empirical question underexplored: when matched-capacity sequence models solve the same controlled task, do they learn aligned internal representations or architecture-specific ones? We answer this question with a compact benchmark that compares 2-layer Transformers, LSTMs, and GRUs on five synthetic tasks spanning arithmetic, formal languages, and retrieval. We train matched models across two random seeds, extract layer activations, and compute cross-architecture and within-architecture centered kernel alignment (CKA) heatmaps. The main result is a mixed picture: modular addition exhibits substantial cross-architecture alignment at high accuracy, Dyck tasks produce much stronger recurrent-to-recurrent alignment than Transformer-to-recurrent alignment, and low-performing retrieval tasks show moderate similarity despite weak accuracy. These results suggest that representational convergence is task dependent rather than universal, and that cross-architecture similarity is most informative when interpreted jointly with performance and within-architecture baselines.

1. Introduction

Mechanistic interpretability seeks to understand how neural networks implement computation using their in-

ternal activations, weights, and circuits. Much of the modern literature has focused on Transformer-specific mechanisms, including induction heads and modular arithmetic circuits (Vaswani et al., 2017; Olsson et al., 2022; Nanda et al., 2023). In contrast, recurrent models are more often discussed through their inductive biases or formal expressive limits than by directly comparing their learned representations to attention-based models (Hochreiter & Schmidhuber, 1997; Cho et al., 2014; Wen et al., 2024).

This asymmetry matters for a basic reason: if different sequence architectures converge to similar internal geometries on the same task, then at least some mechanistic explanations may transfer across model families. If they do not, then architecture-specific tools and theories remain necessary even in small, controlled domains. Behavioral accuracy alone cannot resolve this issue. Two models may solve the same task with similar accuracy while relying on different intermediate representations, or may exhibit similar internal geometry while failing in the same way.

We study this question in a deliberately small but controlled setting. Using a shared codebase, we train matched 2-layer Transformer, LSTM, and GRU classifiers on five synthetic tasks: modular addition, Dyck-1, Dyck-2, induction, and associative recall. We then compute layer-wise centered kernel alignment (CKA) heatmaps (Kornblith et al., 2019) between architectures and compare them to within-architecture seed baselines.

Our main contribution is an empirical benchmark for cross-architecture representational comparison in mechanistic interpretability. The benchmark is small enough to inspect directly, but broad enough to expose qualitatively different regimes. We find three robust patterns. First, modular addition yields high accuracy and substantial cross-architecture alignment, suggesting convergent solutions are possible. Second, Dyck tasks, especially Dyck-2, show a stronger recurrent-to-recurrent alignment than Transformer-to-recurrent alignment, consistent with architecture-specific inductive biases for hierarchical state tracking. Third, in-

¹AUTHORERR: Missing \icmlaffiliation. .AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

duction and associative recall remain difficult for all three models in this setting, yet still show moderate similarity, indicating that representational similarity alone should not be interpreted as evidence of successful mechanistic convergence.

2. Related Work

Our work sits at the intersection of mechanistic interpretability, representation similarity, and sequence-architecture comparison.

Mechanistic interpretability in Transformers. Transformer-focused work has produced some of the clearest mechanistic case studies in the field. Olsson et al. (2022) argue that induction heads are a core mechanism for in-context learning, while Nanda et al. (2023) reverse engineer modular addition circuits in small Transformers and connect them to Fourier structure. These works motivate our task choices and provide concrete hypotheses about when attention-based models may learn distinctive internal algorithms.

Representation similarity methods. CKA has become a standard tool for comparing learned representations across layers and models because it remains stable in settings where CCA-style approaches can fail (Kornblith et al., 2019). At the same time, representation-similarity scores can be confounded by dataset structure, especially in shallow layers (Cui et al., 2022). We therefore interpret our CKA values comparatively, emphasizing cross-architecture patterns together with within-architecture baselines and downstream accuracy rather than treating any single score as mechanistically decisive.

Recurrent models and architectural limits. LSTMs and GRUs were introduced to improve recurrent learning and long-range memory (Hochreiter & Schmidhuber, 1997; Cho et al., 2014). More recent theory sharpens where recurrent and attention-based models differ, especially on retrieval-heavy tasks. In particular, Wen et al. (2024) argue that in-context retrieval is a key bottleneck for recurrent architectures relative to Transformers. Our results complement that line of work by moving from behavioral or theoretical comparison to layer-wise representation comparison. On Dyck-style formal language tasks, prior work also highlights that recurrent memory mechanisms can be especially relevant for hierarchical structure (Suzgun et al., 2019).

Closest gap. To our knowledge, prior work does not provide a compact benchmark of cross-architecture

CKA heatmaps across modular arithmetic, Dyck languages, induction, and associative recall using matched-capacity Transformer, LSTM, and GRU models. The gap is not the existence of CKA, nor the existence of these tasks individually, but the controlled combination of both for direct geometric comparison.

3. Experimental Setup

3.1. Tasks

We evaluate on five synthetic sequence-classification tasks implemented in a shared codebase:

- Modular addition: predict $(a + b) \bmod 10$ from short tokenized equations.
- Dyck-1: binary classification of balanced versus corrupted single-bracket sequences.
- Dyck-2: the same setting with two bracket types.
- Induction: retrieve a value associated with a queried key from a short key-value prompt.
- Associative recall: retrieve a value from a longer key-value list separated by delimiters.

Each task uses 2,048 training examples and 512 validation examples. We fix the dataset seed to 42 for all tasks and evaluate two model seeds, 42 and 43.

3.2. Models and Training

All architectures use 2 layers and hidden dimension 128. The Transformer uses 4 attention heads and sinusoidal positional encodings. LSTM and GRU models use stacked single-layer recurrent blocks. To keep the comparison controlled, all models use the same batch size (64), optimizer (AdamW), learning rate (10^{-3}), weight decay (10^{-4}), and number of epochs (10). We report mean and population standard deviation across the two model seeds.

3.3. Activation Comparison

For each trained model, we collect mean-pooled activations from the embedding output, layer 1, and layer 2 on the validation set. Given activation matrices $X \in \mathbb{R}^{n \times d_x}$ and $Y \in \mathbb{R}^{n \times d_y}$ for the same examples, we compute linear CKA as

$$\text{CKA}(X, Y) = \frac{\langle \tilde{K}_X, \tilde{K}_Y \rangle_F}{\|\tilde{K}_X\|_F \|\tilde{K}_Y\|_F}, \quad (1)$$

where $\tilde{K}_X$ and $\tilde{K}_Y$ are centered Gram matrices (Kornblith et al., 2019). For each task, we average pairwise

heatmaps across seeds for cross-architecture comparisons and compute a within-architecture baseline by comparing the two seeds of the same architecture.

4. Results

4.1. Accuracy Separates Easy, Structured, and Hard Regimes

Table 1 shows three clear regimes. Modular addition is solved perfectly by all architectures. Dyck-1 is nearly solved by all models, while Dyck-2 exposes a sharper architectural difference: the LSTM remains near perfect, the GRU degrades modestly, and the Transformer is clearly worse. Induction and associative recall are difficult for all models under this small-scale setup, with accuracies near chance plus a modest margin.

This accuracy landscape matters because it changes how representation similarity should be interpreted. On solved tasks, similarity may reflect convergent algorithms. On unsolved tasks, similarity may instead reflect convergent failure modes or shallow heuristics.

4.2. Modular Addition Shows the Strongest Cross-Architecture Convergence

Figure 1 shows that modular addition produces the clearest cross-architecture alignment. Across all architecture pairs, embedding layers align strongly and later layers exhibit substantial similarity as well. This is noteworthy because the underlying computational primitives differ sharply between attention and recurrence, yet the resulting geometry is still partially shared. Combined with the perfect accuracies in table 1, this pattern is consistent with convergent learning of a compact algorithmic representation.

This finding resonates with mechanistic work showing that modular arithmetic can induce highly structured representations in small Transformers (Nanda et al., 2023). Our results suggest that at least some of this structure may not be uniquely Transformer-specific at the level captured by CKA.

4.3. Dyck Tasks Expose Architecture-Specific Internal Structure

The Dyck tasks reveal a different pattern. On Dyck-1 and especially Dyck-2, LSTM-GRU similarity is consistently higher than Transformer-recurrent similarity, particularly in later layers (fig. 1). This matches the accuracy pattern: recurrent models perform best, while the Transformer lags most clearly on Dyck-2.

We interpret this as evidence that hierarchical bracket

tracking induces more recurrent-specific solution geometry than modular addition does. In other words, matched external performance is not sufficient to force a shared internal topology, and when performance diverges, the geometry diverges even more strongly. This observation is broadly consistent with the idea that recurrent state updates are a natural fit for stack-like or counter-like structure, even when all models are trained on the same finite synthetic distribution (Suzgun et al., 2019; Wen et al., 2024).

4.4. Low Accuracy Does Not Eliminate Similarity

Induction and associative recall produce the lowest accuracies, yet their heatmaps are not uniformly low (fig. 1). In particular, LSTM and GRU still align fairly strongly with one another, and Transformer-to-recurrent similarity remains moderate in several cells. This is an important cautionary result. Similarity does not imply success.

One plausible interpretation is that all three architectures learn partially overlapping shallow heuristics on these tasks, without discovering the task-general mechanism required for high accuracy. This matches the broader lesson from Cui et al. (2022): geometric similarity measures are informative but not self-interpreting. In our setting, they become much more useful when paired with baselines and performance.

4.5. Within-Architecture Baselines Are Essential

Figure 2 shows that same-architecture seed baselines are generally stronger and cleaner than cross-architecture comparisons, especially on modular addition and the Dyck tasks. This baseline is crucial for interpretation. For example, moderate Transformer-LSTM similarity on induction would look more impressive in isolation than it does when compared with the stronger LSTM-LSTM or GRU-GRU consistency on the same task.

This comparison suggests a practical rule for future cross-architecture interpretability work: cross-family similarity should be evaluated relative to seed baselines, not treated as meaningful on an absolute scale.

5. Discussion

The benchmark points to a simple but useful framing: representational convergence across sequence architectures is task contingent.

When the task admits a compact, easily discovered algorithm, as in modular addition, attention and recurrence can produce partially aligned internal geometry.

Table 1. Validation accuracy across tasks and architectures. Values are mean $\pm$ standard deviation over two model seeds.

Task	Transformer	LSTM	GRU
Modular Addition	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
Dyck-1	0.973 $\pm$ 0.002	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000
Dyck-2	0.940 $\pm$ 0.004	0.999 $\pm$ 0.001	0.956 $\pm$ 0.013
Induction	0.516 $\pm$ 0.010	0.530 $\pm$ 0.011	0.537 $\pm$ 0.006
Associative Recall	0.321 $\pm$ 0.015	0.354 $\pm$ 0.008	0.356 $\pm$ 0.015

When the task favors persistent state tracking, as in Dyck-2, recurrent models align more strongly with one another than with Transformers. When all models perform poorly, as in our induction and associative-recall setting, similarity can still be present, but it is more likely to reflect shared weakness than shared mechanistic competence.

These findings have two implications for mechanistic interpretability. First, the portability of explanations across architectures is likely to be limited and task dependent. Second, geometry-level analyses such as CKA are most valuable when used as a triage tool: they can identify where closer mechanistic study is likely to transfer and where it is likely to fail.

6. Limitations and Future Work

This study is intentionally small scale. We use only two seeds, short synthetic tasks, and compact 2-layer models. Our activation comparison is based on mean-pooled token representations and vanilla linear CKA, which may obscure token-level or causal structure. In addition, CKA can be confounded by shared dataset geometry, particularly in early layers (Cui et al., 2022). For these reasons, our results should be interpreted as a benchmark and set of hypotheses rather than a definitive claim about universal architectural behavior.

Several extensions are straightforward. First, the same pipeline can incorporate larger models and more seeds. Second, future work should pair CKA with causal interventions, probing, or patching analyses to test whether aligned geometry corresponds to aligned mechanisms. Third, modern recurrent and state-space models could reveal whether the patterns observed here extend beyond classical LSTM/GRU baselines.

7. Conclusion

We presented a compact benchmark for comparing Transformer, LSTM, and GRU representations across five synthetic mechanistic-interpretability tasks using cross-architecture and within-architecture CKA heatmaps. The main result is that convergence is selective rather than universal: modular addition yields

clear cross-architecture alignment, Dyck tasks favor recurrent-specific geometry, and low-performing retrieval tasks show that moderate similarity can coexist with weak task competence. We hope this benchmark is useful both as a reproducible artifact and as a reminder that mechanistic explanations should be tested not only within architectures, but across them.

Impact Statement

This paper presents work whose goal is to advance the field of mechanistic interpretability. There are many potential societal consequences of improved understanding of neural-network internals, including both better model debugging and better auditing of failure modes. Because this study is limited to small synthetic tasks, it does not itself enable direct deployment or misuse, but it may contribute modestly to methods for analyzing larger models in future work.

References

- Cho, K., van Merriënboer, B., Gülçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078, 2014.
- Cui, T., Kumar, Y., Marttinen, P., and Kaski, S. Deconfounded representation similarity for comparison of neural networks. In *Advances in Neural Information Processing Systems* 35, 2022.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3519–3529, 2019.
- Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhardt, J. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023.

- Olsson, C., Elhage, N., Nanda, N., Joseph, N., Das-Sarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., et al. In-context learning and induction heads. arXiv preprint arXiv:2209.11895, 2022.
- Suzgun, M., Gehrmann, S., Belinkov, Y., and Shieber, S. M. Memory-augmented recurrent neural networks can learn generalized dyck languages. arXiv preprint arXiv:1911.03329, 2019.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In Advances in Neural Information Processing Systems 30, pp. 5998–6008, 2017.
- Wen, K., Dang, X., and Lyu, K. Rnns are not transformers (yet): The key bottleneck on in-context retrieval. arXiv preprint arXiv:2402.18510, 2024.

Cross-Architecture CKA Heatmaps Across Synthetic Tasks

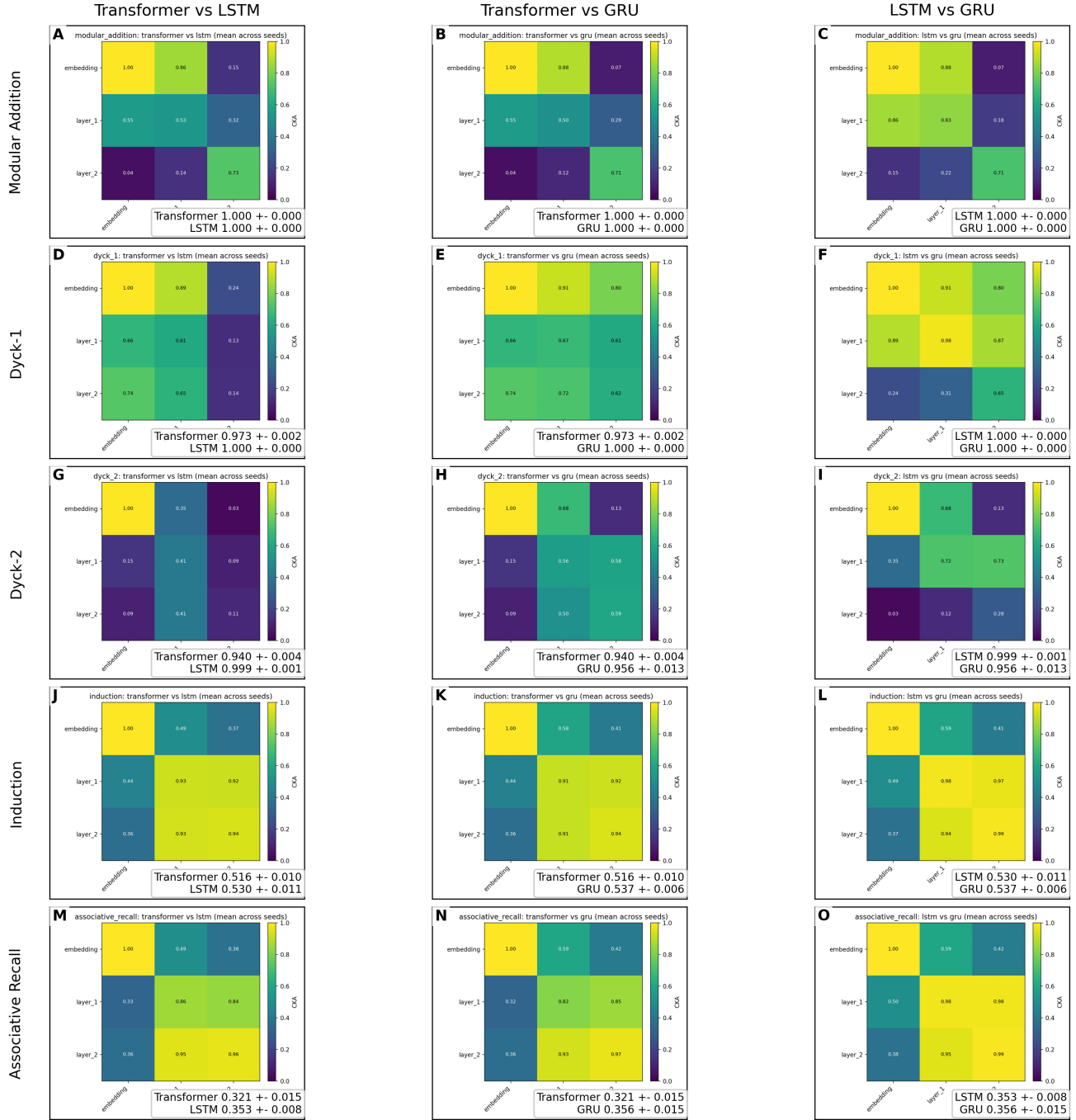


Figure 1. Cross-architecture CKA heatmaps averaged across seeds. Rows correspond to tasks and columns to architecture pairs. Modular addition shows substantial cross-architecture alignment, especially between later layers. Dyck tasks show comparatively weaker Transformer-to-recurrent alignment than recurrent-to-recurrent alignment, while induction and associative recall show moderate similarity despite low accuracy.

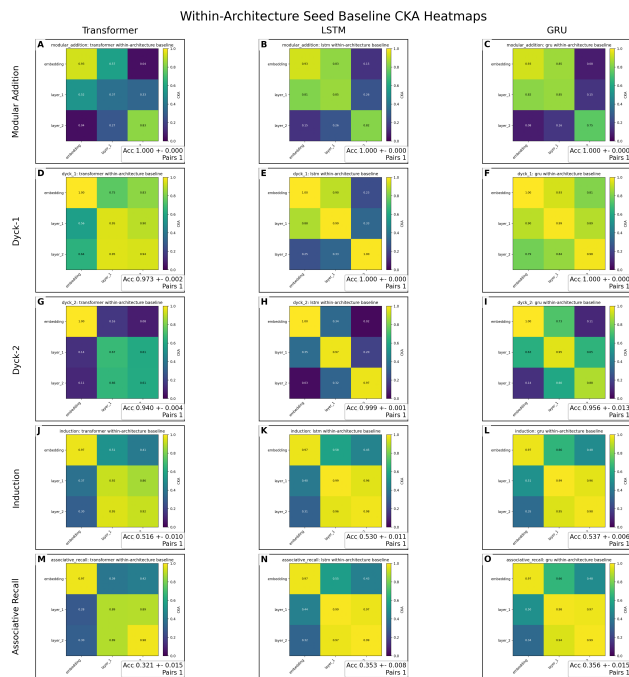


Figure 2. Within-architecture seed-baseline CKA heatmaps. Same-architecture runs are highly self-consistent on modular addition and Dyck tasks, and still moderately self-consistent on the harder retrieval tasks. Cross-architecture comparisons should therefore be read relative to these stronger same-family baselines.

A. Implementation Summary

All experiments were run with a single shared training pipeline. The Transformer uses learned token embeddings, sinusoidal positional encodings, and two encoder layers with GELU feed-forward sublayers. The LSTM and GRU baselines use the same hidden size and depth. For all architectures, layer activations are mean pooled over sequence positions before CKA is computed.

B. Artifact Availability

The paper uses three figures generated from the experiment outputs: a cross-architecture CKA summary, a within-architecture baseline summary, and an accuracy bar chart. The codebase also exports per-task heatmaps and a CSV metrics table, which can support future extensions or appendix material.

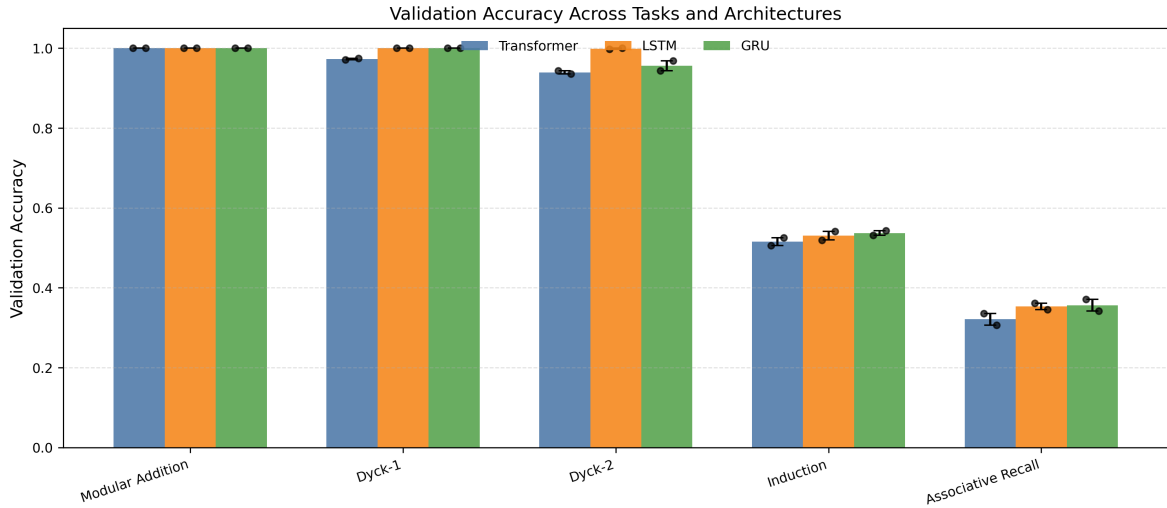


Figure 3. Accuracy summary reproduced in the appendix for convenience. The ranking across tasks is stable across the two seeds, with LSTM best on Dyck-2 and all models weak on induction and associative recall.